



Programmieren

Falk Morgenstern

Leon Johann Brettin

Foliensatz basierend auf Unterlagen von Prof. Huhn, Prof. Meyer und weiteren.
Nur zur Verwendung in der Vorlesung „Programmieren“ an der Ostfalia.
Keine Weitergabe.



Programmieren

7. Generics



Programmieren

7. Generics

7.1 Generische Typen



Generizität: Einführung

- Oft möchte man Klassen, Interfaces oder Methoden zu einem beliebigen, aber festen Datentyp schreiben (Instanzvariable, Übergabeparameter etc.)
- Beispiel: `Stack` (Stapelspeicher) zur Speicherung beliebiger Objekte
 - `int`-Objekte
 - `String`-Objekte
 - `MeineKlasse`-Objekte
- Hierzu werden bei der Definition der Klassen, Interfaces oder Methoden formale **Typparameter** (Platzhalter) eingeführt
 - Repräsentieren zum Zeitpunkt der Implementierung einen noch unbekannten Typ
 - Werden bei der Verwendung durch einen konkreten Typ ersetzt
 - Man definiert „Schablonen“, die erst durch Angabe von konkreten Typen bei ihrer Verwendung zu normalen Klassen, Interfaces oder Methoden werden
- Diese Möglichkeit heißt Generizität (**Generics**)



Generizität: Einführung

- Generizität gibt es seit Java 5.
- Sie erlaubt, Klassen, Interfaces und Methoden mit Typen zu parametrisieren.
- Der Name des Typparameters wird in spitzen Klammern an den Namen der Klasse / des Interface angehängt bzw. der Methodensignatur angegeben
- Collections-Typen (siehe folgende Kapitel) sind generische Klassen!
- Statt eine Liste beliebiger Objekte zu implementieren, kann man nun implementieren:
 - Liste von Personen
 - Liste von Namen
 - Liste von Artikeln
 - ...



Beispiel: generische Liste mit Typparameter T

```
public class GenericList<T> {  
    private final int SIZE = 10;  
    private T[] data = (T[]) new Object[SIZE]; // ?? später!  
    private int index = 0;  
  
    public void add(T x) { // haengt hinten an die Liste an  
        if (index < SIZE) {  
            data[index++] = x;  
        }  
    }  
  
    public T remove() { // entfernt das letzte Element der Liste  
        if (index > 0) {  
            return data[--index];  
        } else {  
            return null;  
        }  
    }  
}
```





Beispiel: Verwendung der generische Liste

```
public static void main(String[] args) {  
    GenericList<Integer> ildist = new GenericList<Integer>();  
    ildist.add(1);  
    ildist.add(2);  
    System.out.println(ildist.remove()); // 2  
    System.out.println(ildist.remove()); // 1  
    System.out.println(ildist.remove()); // null  
  
    GenericList<String> slist = new GenericList<String>();  
    slist.add("A");  
    slist.add("B");  
    System.out.println(slist.remove()); // "B"  
    System.out.println(slist.remove()); // "A"  
    System.out.println(slist.remove()); // null  
}
```





Erläuterungen: GenericList<T>

- **Typparameter** T ist ein Platzhalter für einen Referenztypen
- T wird in spitzen Klammern hinter den Klassennamen geschrieben
- T steht innerhalb der Klasse dort, wo der zu ersetzende Typ stehen würde
- Bei der Variablendeklaration und Objekterzeugung wird der konkrete Referenztyp, der für T stehen soll, angegeben („**parametrisierte Klasse**“)
- Alle Referenztypen sind erlaubt (Klassen und Interfaces; verkürzt: „Klassen“)
- Primitive Typen (int, float, ...) sind nicht erlaubt (aber Auto-Boxing / Unboxing möglich)
- Wenn der Compiler den korrekten Typ feststellen kann, reicht <> („Diamond“)

```
GenericList<Integer> ildist = new GenericList<Integer>();  
GenericList<String>  slist = new GenericList<String>();  
GenericList<String>  tlist = new GenericList<>();
```





Vorteile generischer Typen

1. Compiler kann gezielte Typprüfungen durchführen. Bei `GenericList<String>` stellt er sicher, dass nur `String`-Objekte verwendet werden können.
 2. Programme werden lesbarer. Man muss keine Typkonversionen durchführen. Es ist klarer sichtbar, welche Objekte in der Liste abgespeichert werden sollen
- Generische Klasse = Klasse mit formalen Typparametern
 - Konkreter Typ = Ausgeprägter Typ = Generische Klasse mit angegebenen Typen = parametrisierte Klasse



Generische Typen und Java

- Generische Typen wurden **nachträglich** in Java hinzugefügt
 - Dabei wurde die Syntax von Java erweitert
 - Die JVM und das Byte-Code-Format wurden aber nicht erweitert (Grund: Abwärtskompatibilität beim WORA-Prinzip)
 - Generische Klassen werden im Byte-Code also auf nicht-generische Klassen abgebildet
 - Die Typinformation wird daher beim Compilieren gelöscht (**Type Erasure**)
 - der Compiler soll dennoch zur Compile-Zeit garantieren, dass zur Laufzeit keine Typfehler auftreten



Rohtypen und Zuweisungskompatibilität

- Compiler erzeugt eine `class`-Datei und ersetzt dabei den Typparameter `T` durch `Object` („type erasure“)
- Der generische Typ (`GenericList<T>`) wird in den Rohtyp (`GenericList`) übersetzt
- Zur Laufzeit ist nur der Rohtyp bekannt
- Folgende Zuweisung ist zulässig: `GenericList rawList = new GenericList<String>();`

```
public class GenericList<T> {  
    private T[] data = // ...  
    public void add(T x) { // ...  
    public T remove() { // ...  
}
```

-->

```
public class GenericList {  
    private Object[] data = // ...  
    public void add(Object x) { // ...  
    public Object remove() { // ...  
}
```



Rohtypen und Zuweisungskompatibilität

- Über parametrisierten Typen lassen sich keine anderen Typen der Liste hinzufügen:

```
GenericList<Integer> ildist = new GenericList<Integer>();  
ildist.add(42);  
ildist.add("Hello"); // Compilefehler  
ildist.add(0.0f);    // Compilefehler
```





Rohtypen und Zuweisungskompatibilität

- Zwei verschieden ausgeprägte generische Typen sind nicht zuweisungskompatibel:

```
GenericList<Integer> intList = new GenericList<String>(); // Compilefehler
```



- Auch wenn die generischen Typen in Unterklassenbeziehung zueinander stehen

```
GenericList<Object> intList = new GenericList<String>(); // Compilefehler
```





Generische Typen und Arrays

- Java: Verbot der Instanziierung von Arrays generischer Typen
 - `T[] data = new T[SIZE];` führt zu Compilerfehler.
- Erklärung
 - Wäre die Deklaration möglich, würde die Typsicherheit gefährdet (durch die Kovarianz des Arrays)
- Workaround (aber nicht typsicher)
`T[] data`
`= (T[]) new Object[SIZE];`

```
public class GenericArrayTest<T> {  
  
    T[] x = new T[10]; //angenommen, das geht  
  
    T[] getX() { return x; }  
  
    public static void main(String[] args) {  
        GenericArrayTest<Integer> s  
            = new GenericArrayTest<Integer>();  
        Number[] n = s.getX();  
        n[0] = 5.0f; // compilierbar,  
                    // würde zu Laufzeitausnahme führen  
    }  
}
```



Mehrere Typen

- Es können auch mehrere Typen mit entsprechenden Platzhaltern angegeben werden

```
public class MixedTriple<T, U, V> {  
  
    T content0; U content1; V content2;  
  
    public MixedTriple(T t, U u, V v) {  
        content0 = t;  
        content1 = u;  
        content2 = v;  
    }  
  
    public static void main(String[] args) {  
        MixedTriple<String, Integer, Person> mt =  
            new MixedTriple<>("Hallo", 12, new Person());  
    }  
}
```



Programmieren

7. Generics

7.2 Einschränkungen bei Typparametern



Eingeschränkte Typparameter

- Bisher werden in `GenericList<T>` Listenelemente nur abgespeichert und ausgelesen.
- Da wir nichts über `T` wissen, können wir nur die Methoden anwenden, die in der Klasse `java.lang.Object` definiert sind.
- Möchte man Operationen auf den generischen Typen anwenden, reicht `Object` als Typinformation oft nicht.
- Um eine spezielle Methode auf `T` anwenden zu können, müssen wir den Typparameter `T` entsprechend einschränken: `T extends ...`
- Dadurch wird `T` auf einen bestimmten Obertyp (Klasse oder Interface) eingeschränkt, so dass die Methoden dieses Obertyps zur Verfügung stehen



Beispiel mit Oberklasse Number

- Ein generischer Typ NumberBox erlaubt, seinen Inhalt als double zurückzuliefern.
- Dazu muss T von Number erben

```
public class NumberBox<T extends Number> {  
    private T content;  
  
    public T getContent() { return content; }  
  
    public double asDouble() {  
        return content.doubleValue();  
    }  
}
```





Beispiel sortierte Liste

- Liste soll in (aufsteigend) sortierter Reihenfolge gespeichert werden.
- Dazu benötigen wir die Methode `compareTo(T x)`, die im Interface `Comparable<T>` deklariert ist
- Wir fordern also, dass `T` das Interface `Comparable<T>` implementiert.



Anforderungen an compareTo

- Ergebnis von `x.compareTo(y)`:
 - < 0 , falls $x < y$
 - > 0 , falls $x > y$
 - $= 0$, falls $x = y$
- `compareTo` soll transitiv sein:
 - $x.compareTo(y) > 0$ und $y.compareTo(z) > 0$ impliziert $x.compareTo(z) > 0$
- `compareTo` soll konsistent sein zu `equals`:
 - $x.compareTo(y) == 0$ genau dann, wenn $x.equals(y)$
- Mögliche Ausnahme: `ClassCastException`
- Frage: Warum wäre eine Inkonsistenz mit `equals` problematisch?
Frage: Wer löst gegebenenfalls die `ClassCastException` aus?



Beispiel sortierte Liste



```
public class SortedList<T extends Comparable<T>> {  
    private final int SIZE = 100;  
    private T[] data = (T[]) new Comparable[SIZE];  
    private int index = 0;  
  
    public void add(T x) {  
        if (index < SIZE) {  
            int i = index - 1;  
            while (i >= 0 &&  
                data[i].compareTo(x) >= 0) {  
                data[i + 1] = data[i];  
                i--;  
            }  
            data[i + 1] = x;  
            index++;  
        }  
    }  
    // ...  
}
```



```
public T remove() {  
    if (index > 0) {  
        return data[--index];  
    } else { return null; }  
}  
  
public static void main(String[] args) {  
    SortedList<Integer> sortList = new  
        SortedList<Integer>();  
  
    sortList.add(7);  
    sortList.add(4);  
    sortList.add(0);  
    sortList.add(3);  
    System.out.println(sortList.remove()); // 7  
    System.out.println(sortList.remove()); // 4  
    System.out.println(sortList.remove()); // 3  
    System.out.println(sortList.remove()); // 0  
  
}
```



Anmerkungen sortierte Liste

- Bei der Erzeugung des Objekts `sortList` stellt der Compiler sicher, dass der Typparameter `Integer` das Interface `Comparable` implementiert.
- Wir erzeugen in `SortedList<T>` das Array `data` mit `new Comparable[SIZE]` statt `new Object[SIZE]`.
- Bei `<T extends X>` muss man als Elementtyp des Arrays `X` statt `Object` verwenden.
- Grund: Im Rohtyp wird `T` nun durch `X` ersetzt und nicht mehr durch `Object`.



Mehrere Einschränkungen des Typparameters

- Typparameter können durch mehrere Typen (Interfaces oder Klassen) eingeschränkt werden.
- Hier muss der Typparameter T Unterklasse der Klasse `MyClass` sein und das Interface `Comparable<T>` implementieren.

```
public class MyList<T extends MyClass & Comparable<T>> {  
    // ...  
}
```





Programmieren

7. Generics

7.3 Vererbung



Generizität und Vererbung

- Generische Klassen werden bei der Vererbung wie gewöhnliche Klassen behandelt.
 - Generische Klassen können Oberklasse oder Unterklasse anderer Klassen sein.
 - Analog für generische Interfaces.
-
- Frage: Kennen Sie Beispiele für generische Interfaces?



Generizität und Vererbung

- Generische Klassen können Basisklasse für Vererbung sein

```
public class A0 extends B0<String>{ /*...*/ }
```



```
public class A0 extends B0<T>{ /*...*/ } // nicht erlaubt, T muss konkret sein
```

- Eine generische Klasse darf von folgenden Klassen erben

```
// von einer nicht generischen Klasse
```

```
public class A1<T> extends B1 { /* ... */ }
```



```
// von einer konkretisierten generischen Klasse
```

```
public class A2<T> extends B2<String> { /* ... */ }
```

```
// von einer generischer Klasse mit gleicher Typvariable
```

```
public class A3<T> extends B3<T> { /* ... */ }
```



Regeln für Zuweisungskompatibilität

- Da `A1<T>` Unterklasse von `B1` ist, dürfen Objekte aller aus `A1<T>` erzeugten konkreten Klassen an Variablen vom Typ `B1` zugewiesen werden



```
public class A1<T> extends B1 {  
  
    public static void main(String[] args) {  
        B1 b1 = new A1<Integer>();  
        B1 b2 = new A1<Person>();  
    }  
}
```



Regeln für Zuweisungskompatibilität

- Alle aus `A2<T>` erzeugten Typen sind mit `B2<String>` kompatibel

```
public class A2<T> extends B2<String> { /* ... */ }
```





Regeln für Zuweisungskompatibilität

- Alle aus A3<T> erzeugten Klassen sind nur dann mit B3<T> kompatibel, wenn die aktuellen Typparameter übereinstimmen.

```
public class A3<T> extends B3<T> { /* ... */ }
```



```
public static void main(String[] args) {  
    B3<Integer> b5 = new A3<Integer>(); // kompatibel  
    B3<Integer> b6 = new A3<Short>();   // nicht kompatibel  
}
```





Überschreiben von Methoden

- Überschreiben einer Methode, die von einem konkretisierten generischen Typ vererbt wurde

```
public class MyList extends GenericList<Integer> {  
    @Override  
    public void add(Integer x) { super.add(x); }  
}
```



- Überschreiben einer Methode, die von einem nicht konkretisierten generischen Typ vererbt wurde

```
public class MyList2<T> extends GenericList<T> {  
    @Override  
    public void add(T x) { /* ... */ }  
}
```





Laufzeitprüfung und Typkonversion

- Generische Typen, wie `GenericList<String>` oder `GenericList<Integer>`, existieren in Java nur zur Compilezeit.
- Zur Laufzeit werden sie wie Rohtypen (ggf. mit implizit eingefügtem Type Cast) behandelt.
- Daher sind Typprüfungen zur Laufzeit nur eingeschränkt möglich.

```
Object obj = new GenericList<String>();  
// obj enthält ein Objekt im Rohtyp  
if(obj instanceof GenericList<String>) { // verboten  
}  
if(obj instanceof GenericList ) { // Typprüfung Rohtyp erlaubt  
}
```





Laufzeitprüfung und Typkonversion

- Typkonversionen sind zwar möglich, aber Compiler liefert Warnung.
- Zur Laufzeit kann nicht geprüft werden, ob `obj` vom Typ `GenericList<String>` ist.
- Zur Laufzeit kann nur geprüft werden, ob `obj` zum Rohtyp `GenericList` gehört.

```
GenericList<String> list = (GenericList<String>) obj;  
// Verantwortung des Entwicklers
```





Invarianz

- Aus „A ist kompatibel zu B“ folgt nicht „C<A> ist kompatibel zu C“
- D.h. die Kompatibilität der Typargumente überträgt sich nicht auf die parametrisierten Typen: Invarianz

```
NumberBox<Number> b = new NumberBox<Integer>(); // nicht erlaubt
```



- Damit wird verhindert, dass Objekte, die nicht vom Typ Integer sind, durch bloße „Umetikettierung“ der Box hinzugefügt werden können
- Bei Arrays in Java ist genau das möglich.



Programmieren

7. Generics

7.4 Wildcard-Typen



Zuweisungskompatibilität

- Ein konkretisierter generischer Typ ist mit keinem anderen generischen Typ zuweisungskompatibel (Invarianz)
- Zuweisung zum Rohtyp ist möglich

```
GenericList<Object> list1 = new GenericList<Integer>(); // Compilerfehler  
GenericList list2 = new GenericList<Integer>(); // Rohtyp funktioniert
```





Wildcard-Typen

- Statt des Rohtyps können wir den Wildcard-Typ `GenericList<?>` verwenden.
- Der Typparameter `T` wird durch das Zeichen `?` ersetzt.
- Jede Typ-Variante, die aus `GenericList<T>` erzeugt wird, ist mit `GenericList<?>` kompatibel.
- Sprechweise: `GenericList` of unknown

```
GenericList<?> list3 = new GenericList<Integer>();  
GenericList<?> list4 = new GenericList<String>();
```





Wildcard-Typen

- Sind Rohtypen und Wildcard-Typen gleich? Nein!
- Beim Rohtyp wird jedes Vorkommen von T durch Object (bzw. durch die Typeinschränkung, den „Bound“ X) ersetzt.
- Beim Wildcard-Typ ist die Belegung von T bewusst unknown / unbekannt.

```
GenericList rawList = new GenericList<Integer>();  
rawList.add("s"); // funktioniert  
rawList.add(1);   // funktioniert  
Object rObj = rawList.remove(); // funktioniert  
  
GenericList<?> wildList = new GenericList<Integer>();  
wildList.add("s"); // Compilerfehler  
wildList.add(1);   // Compilerfehler  
Object wObj = wildList.remove(); // funktioniert
```





Eingeschränkte Wildcard-Typen

- Nutzen von Wildcard-Typen:
 - Typparameter T in der Klassendeklaration lässt sich bei der Deklaration von Objekten (bei der eine konkrete Typangabe für T erforderlich ist) auf Unterklassen eines gegebenen Basistyps einschränken.
 - Beispiel: Oberklasse Number, Unterklassen Double, Integer

```
GenericList<? extends Number> numberList; // konkrete Typangabe;  
                                           // daher kein T, sondern ?  
numberList = new GenericList<Double>();  
numberList = new GenericList<Integer>();
```





Verwendung eingeschränkter Wildcard-Typen

- Was wissen wir über die Variable `numberList`?
 - Der Typ der Elemente der Liste ist weiter unknown, erweitert aber die Klasse `Number`.
 - Lesezugriff über `numberList` ist möglich, Typ `Number` ist zugesichert.
 - Schreibzugriff über `numberList` mit `add`-Methoden funktioniert nicht
 - hierzu ist eine Liste mit konkretem Typparameter (z.B. `doubleList`) nötig.
- Idee ist also:
 - Zunächst eine Liste mit konkretem Typ generieren und erst dann an `numberList` zuweisen.
 - Es können hierauf nur Leseoperationen durchgeführt werden.



Wildcards: Beispiel „Upper Bounded Wildcard“

- `GenericList` wird um eine `addAll`-Methode erweitert, mit der Elemente einer weiteren `GenericList` hinzugefügt werden können
- Erster Versuch

```
public void addAll(GenericList<T> list) {  
    for (int i = 0; i < list.getLength(); i++) {  
        this.add(list.get(i));  
    }  
}  
// ...
```



```
GenericList<Integer> iL = new GenericList<>();  
iL.add(1); iL.add(2);  
GenericList<Number> nL = new GenericList<>();  
nL.add(1);  
nL.addAll(iL); // Compilefehler
```





Wildcards: Beispiel „Upper Bounded Wildcard“

- Verbesserung

```
public void addAll(GeneriList<? extends T> list) {  
    for (int i = 0; i < list.getLength(); i++) {  
        this.add(list.get(i));  
    }  
}  
// ...
```



```
GeneriList<Integer> i1 = new GeneriList<>();  
i1.add(1); i1.add(2);  
GeneriList<Number> n1 = new GeneriList<>();  
n1.add(1);  
n1.addAll(i1); // jetzt ok
```





Wildcards: Beispiel „Lower Bounded Wildcard“

- `GenericList` wird um eine `removeIf`-Methode erweitert, die mit Hilfe eines Prädikats alle Element prüft und ggf. löscht, wenn das Prädikat zutrifft
- Erster Versuch

```
// ...  
public void removeIf(ListPredicate<T> tester) {  
    for (int i = 0; i < index; i++) {  
        if (tester.test(this.get(i))) {  
            this.remove(i);  
        }  
    }  
}  
// ...
```





Wildcards: Beispiel „Lower Bounded Wildcard“

- Erster Versuch

```
public interface ListPredicate<T> {  
    public boolean test(T t);  
}
```



```
public class ListPredA implements ListPredicate<Number> {  
    public boolean test(Number t) {  
        if (t.intValue() == 1) { return true; }  
        return false;  
    }  
}
```



```
GenericList<Integer> il = new GenericList<>();  
ListPredicate<Number> pred = new ListPredA();  
il.removeIf(pred); // Compileerror
```





Wildcards: Beispiel „Lower Bounded Wildcard“

- Verbesserung

```
// ...  
public void removeIf(ListPredicate<? super T> tester) {  
    for (int i = 0; i < index; i++) {  
        if (tester.test(this.get(i))) {  
            this.remove(i);  
        }  
    }  
}  
// ...
```



```
GenericList<Integer> il = new GenericList<>();  
ListPredicate<Number> pred = new ListPredA();  
il.removeIf(pred); // jetzt o.k.
```





Programmieren

7. Generics

7.5 Generische Methoden



Generische Methoden

- Nicht nur Klassen, sondern auch Methoden können Typparameter enthalten.
- Dann kann eine Methode auf Daten unterschiedlicher Typen angewendet werden.
- Kann nützlich sein für statische Hilfsfunktionen.

```
public static <T> T random(T m, T n) {  
    return Math.random() > 0.5 ? m : n;  
}
```





Generische Methoden

- Die Methode kann auf Werte desselben Typs angewendet werden, wobei der Typ jedoch beliebig sein kann.
- Beim Aufruf der Methode leitet der Compiler den aktuellen Typparameter aus den Argumenten ab.

```
static <T extends Comparable<T>> T max(T x, T y) {  
    if (x.compareTo(y) > 0) {  
        return x;  
    } else {  
        return y;  
    }  
}
```



```
String s = max("John", "Liza");  
int m = max(5,3);
```





Generische Methoden

- Alternative: Das Interface als Parametertyp verwenden, dann casten

```
static Comparable max2(Comparable x, Comparable y) {  
    if (x.compareTo(y) > 0) {  
        return x;  
    } else {  
        return y;  
    }  
}
```



```
int m2 = (int)max2(5,3);
```





Übung

- Erweitern Sie die Klasse `Kontakt` aus der letzten Übung
 - Implementieren Sie eine Methode `random(Kontakt k)`, die mit einer Wahrscheinlichkeit von 50% den übergebenen Kontakt `k` oder `this` liefert.
 - Lagern Sie die Methode `random` in eine Hilfsklasse aus. Dabei soll die statische Methode auf beliebigen Typen `T` funktionieren.



Übung

- Welche der gezeigten Statements sind erlaubt, welche führen zu Compilefehlern?
 - ...



Übung

- Definieren Sie die generische Klasse `Paar<T,U>`, die Werte zweier unterschiedlicher Typen aufnehmen kann. Über passende `get`-Methoden sollen diese Werte abgefragt werden können.
- Definieren Sie das generische Interface `Markierbar<S>`, das die beiden folgenden Methoden deklariert:
 - `void setMarke(S m)`
 - `S getMarke()`
- Die Klasse `Paar<T,U>` soll dieses Interface mit dem Typargument `String` implementieren.
- Definieren Sie die generische Klasse `ZahlenPaar`, die von der Klasse `Paar<T,U>` abgeleitet ist und deren Typargumente zu `Number` kompatibel sein müssen. Die abstrakte Klasse `Number` enthält u. a. die abstrakte Methode `double doubleValue()`, die z. B. von den Klassen `Integer`, `Double` und `BigInteger` implementiert wird. Die Klasse `ZahlenPaar` soll eine Methode enthalten, die die Summe der beiden Zahlen des Paares als `double`-Zahl zurückgibt.